

**LAPORAN TUGAS KECERDASAN BUATAN
IMPLEMENTASI ALGORITMA A* (A-STAR)
PADA APLIKASI PENCARIAN RUTE TERCEPAT SmarthPath**



Disusun oleh:

1. Ragil Kurniawan

G1A024063

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.TI, Ph.D.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU**

2026

DAFTAR ISI

BAB I	4
LANDASAN TEORI	4
1.1 Teori Graf dan Jaringan Jalan	4
1.2 Algoritma Pencarian Jalur	4
1.3 Algoritma A* (A-Star)	5
1.4 Fungsi Heuristik dalam Pencarian Rute Geografis	6
1.5 Teknologi Pendukung	6
BAB II	8
PEMBAHASAN	8
2.1 Deskripsi Umum Sistem SmartPath	8
2.2 Desain Algoritma A* pada SmartPath	8
2.3 Implementasi	9
2.4 Antarmuka Pengguna	10
2.5 Hasil Pengujian	11
BAB III	13
KESIMPULAN DAN SARAN	13
3.1 Kesimpulan	13
3.2 Saran	13

DAFTAR GAMBAR

Gambar 1. Tampilan antarmuka awal aplikasi SmartPath	7
Gambar 2. Hasil pencarian rute terpendek pada peta Kota Bengkulu	9

BAB I

LANDASAN TEORI

1.1 Teori Graf dan Jaringan Jalan

Graf merupakan struktur matematika yang terdiri dari himpunan simpul (*vertex*) dan himpunan sisi (*edge*) yang menghubungkan pasangan simpul tersebut. Dalam konteks jaringan jalan, simpul merepresentasikan persimpangan atau titik-titik koordinat geografis, sedangkan sisi merepresentasikan ruas jalan yang menghubungkan dua persimpangan. Graf berarah (*directed graph*) digunakan untuk jalan satu arah, sementara graf tidak berarah (*undirected graph*) merepresentasikan jalan dua arah (Rachmawati & Gustin, 2020).

Representasi graf jaringan jalan dalam sistem informasi geografis umumnya menggunakan format *GeoJSON* (Geographic JavaScript Object Notation), yaitu format standar berbasis *JavaScript Object Notation* (JSON) untuk merepresentasikan fitur-fitur geografis beserta atributnya. Setiap ruas jalan direpresentasikan sebagai objek *LineString* yang memuat koordinat-koordinat geografis berupa pasangan nilai *longitude* dan *latitude* (Butler et al., 2022).

1.2 Algoritma Pencarian Jalur

Algoritma pencarian jalur (*pathfinding algorithm*) adalah prosedur komputasi yang bertujuan menemukan jalur dari suatu simpul asal ke simpul tujuan dalam sebuah graf. Algoritma-algoritma klasik yang sering digunakan dalam pencarian jalur antara lain algoritma Dijkstra, algoritma *Breadth-First Search* (BFS), algoritma *Depth-First Search* (DFS), dan algoritma A* (*A-Star*). Di antara algoritma-algoritma tersebut, algoritma A* dikenal sebagai salah satu yang paling efisien karena menggabungkan pendekatan pencarian berbasis biaya aktual dengan fungsi heuristik yang memperkirakan biaya menuju tujuan (Cazenave et al., 2023).

Algoritma Dijkstra menjamin penemuan jalur terpendek, namun memiliki kompleksitas waktu $O(V^2)$ atau $O((V + E) \log V)$ bergantung pada implementasinya, di mana V adalah jumlah simpul dan E adalah jumlah sisi. Keterbatasan algoritma Dijkstra adalah tidak memanfaatkan informasi arah tujuan sehingga mengeksplorasi simpul-simpul yang tidak relevan. Algoritma A*

mengatasinya dengan menambahkan fungsi heuristik sehingga eksplorasi lebih terarah dan efisien (Nugraha et al., 2021).

1.3 Algoritma A* (A-Star)

Algoritma A* pertama kali diperkenalkan oleh Hart, Nilsson, dan Raphael pada tahun 1968 dan hingga kini tetap menjadi algoritma pencarian jalur yang paling banyak digunakan dalam berbagai aplikasi, mulai dari permainan video hingga sistem navigasi berbasis *Geographic Information System* (GIS). Prinsip dasar algoritma A* adalah mengevaluasi setiap simpul yang akan dikunjungi menggunakan fungsi evaluasi sebagai berikut (Zhou & Yang, 2022):

$$f(n) = g(n) + h(n)$$

Keterangan persamaan di atas adalah sebagai berikut: $f(n)$ merupakan nilai fungsi evaluasi total untuk simpul n yang digunakan sebagai dasar prioritas eksplorasi; $g(n)$ adalah biaya aktual (*actual cost*) dari simpul awal hingga simpul n yang telah dihitung secara tepat berdasarkan bobot sisi yang dilalui; dan $h(n)$ adalah nilai heuristik (*heuristic value*) yang merupakan estimasi biaya dari simpul n menuju simpul tujuan. Kualitas algoritma A* sangat bergantung pada kualitas fungsi heuristik yang digunakan (Rachmawati & Gustin, 2020).

Algoritma A* bekerja dengan mempertahankan dua struktur data utama, yaitu *open list* dan *closed list*. *Open list* adalah antrian prioritas (*priority queue*) yang menyimpan simpul-simpul yang sudah ditemukan tetapi belum sepenuhnya dieksplorasi, diurutkan berdasarkan nilai $f(n)$ terkecil. *Closed list* menyimpan simpul-simpul yang sudah sepenuhnya dieksplorasi. Langkah-langkah algoritma A* secara umum adalah sebagai berikut:

1. Masukkan simpul awal ke dalam open list dengan nilai $g = 0$ dan $h =$ estimasi jarak ke tujuan.
2. Ambil simpul n dengan nilai $f(n)$ terkecil dari open list.
3. Jika n adalah simpul tujuan, rekonstruksi jalur dan hentikan pencarian.
4. Pindahkan n ke closed list. Untuk setiap tetangga n' dari n yang belum ada di closed list, hitung $g(n') = g(n) + \text{cost}(n, n')$ dan $h(n')$. Jika n'

belum ada di open list atau nilai g baru lebih kecil, perbarui data n' dan masukkan ke open list.

5. Ulangi dari langkah 2 hingga open list kosong (jalur tidak ditemukan) atau tujuan tercapai.

1.4 Fungsi Heuristik dalam Pencarian Rute Geografis

Untuk pencarian rute pada data geografis nyata, fungsi heuristik yang paling umum digunakan adalah formula *Haversine* yang menghitung jarak busur besar (*great-circle distance*) antara dua titik pada permukaan bumi berdasarkan koordinat lintang dan bujur. Formula *Haversine* menghasilkan estimasi jarak yang *admissible* (tidak pernah melebihi jarak sebenarnya karena tidak mempertimbangkan kelengkungan jalan), sehingga menjamin algoritma A^* menemukan jalur yang optimal (Cazenave et al., 2023).

Formula *Haversine* secara matematis dinyatakan sebagai berikut. Diberikan dua titik koordinat (φ_1, λ_1) dan (φ_2, λ_2) di mana φ adalah lintang dan λ adalah bujur dalam satuan radian, maka jarak *great-circle* $d = 2r \times \arcsin(\sqrt{(\sin^2(\Delta\varphi/2) + \cos(\varphi_1) \times \cos(\varphi_2) \times \sin^2(\Delta\lambda/2))})$, di mana r adalah jari-jari rata-rata bumi sebesar 6.371 kilometer (Zhou & Yang, 2022).

1.5 Teknologi Pendukung

Pengembangan sistem SmartPath memanfaatkan sejumlah teknologi perangkat lunak modern. Di sisi *backend*, digunakan *runtime* Node.js dengan *framework* Express.js sebagai server HTTP. Pustaka *geojson-path-finder* menyediakan implementasi algoritma A^* yang dioptimalkan untuk data *GeoJSON* spasial, sementara *Turf.js* digunakan untuk komputasi geospasial seperti snap-to-road dan kalkulasi jarak (Nugraha et al., 2021).

Di sisi *frontend*, aplikasi dibangun menggunakan *framework* React.js dengan *build tool* Vite dan *framework* CSS Tailwind. Visualisasi peta menggunakan *library* Leaflet.js melalui *wrapper* React-Leaflet, dengan data ubin peta bersumber dari CartoDB Voyager. Layanan *geocoding* memanfaatkan API Nominatim dari OpenStreetMap untuk mengubah nama tempat menjadi koordinat geografis (OpenStreetMap Contributors, 2023).

Data jaringan jalan diperoleh melalui Overpass API milik OpenStreetMap, yaitu sebuah *read-only API* yang memungkinkan kueri data peta secara terstruktur menggunakan bahasa Overpass QL. Data yang diperoleh dikonversi dari format OSM JSON ke format *GeoJSON* menggunakan pustaka *osmtogeojson* sebelum disimpan sebagai graf spasial yang siap diolah oleh algoritma A* (Butler et al., 2022).

BAB II

PEMBAHASAN

2.1 Deskripsi Umum Sistem SmartPath

SmartPath adalah aplikasi web pencarian rute tercepat yang dibangun dengan arsitektur *client-server* terpisah antara *frontend* dan *backend*. Sistem ini secara khusus dirancang untuk melakukan pencarian jalur terpendek pada jaringan jalan nyata di wilayah Kota Bengkulu menggunakan algoritma A* yang diimplementasikan secara murni di sisi *backend*. Pengguna dapat menentukan titik awal dan titik tujuan dengan cara mengeklik langsung pada peta interaktif atau mengetikkan nama lokasi pada kolom pencarian yang terhubung dengan layanan *geocoding* Nominatim.

Sistem memiliki dua komponen utama yang berjalan secara terpisah. Komponen *backend* bertanggung jawab atas seluruh logika komputasi rute, termasuk pemuatan graf jaringan jalan, proses *snap-to-road* (penjepitan titik ke jalan terdekat), eksekusi algoritma A*, serta kalkulasi jarak dan estimasi waktu tempuh. Komponen *frontend* bertanggung jawab atas antarmuka pengguna yang interaktif, meliputi tampilan peta, visualisasi rute, dan penyajian informasi hasil pencarian kepada pengguna.

2.2 Desain Algoritma A* pada SmartPath

Desain algoritma A* dalam sistem SmartPath mengadaptasi implementasi standar algoritma A* terhadap konteks data geografis nyata. Terdapat beberapa penyesuaian desain yang diterapkan agar algoritma dapat bekerja secara efektif pada jaringan jalan sesungguhnya.

Pertama, representasi graf menggunakan data *GeoJSON* yang bersumber dari OpenStreetMap. Setiap fitur *LineString* dalam *GeoJSON* mewakili satu ruas jalan dengan serangkaian koordinat *longitude-latitude*. Pustaka *geojson-path-finder* secara otomatis mengonversi data *GeoJSON* tersebut menjadi graf topologi di mana simpul-simpul adalah titik-titik koordinat dan sisi-sisinya adalah segmen jalan dengan bobot berupa jarak Euclidean atau *Haversine* antar simpul (Cazenave et al., 2023).

Kedua, proses *snap-to-road* dilakukan sebelum algoritma A* dijalankan. Karena koordinat yang diberikan pengguna melalui klik pada peta kemungkinan besar tidak tepat jatuh pada simpul graf jaringan jalan, sistem menggunakan fungsi *turf.nearestPoint()* dari pustaka Turf.js untuk menemukan titik koordinat yang paling dekat dengan koordinat input pengguna di antara seluruh simpul yang ada dalam graf. Proses ini memastikan bahwa titik awal dan tujuan selalu berada pada jaringan jalan yang valid.

Ketiga, bobot sisi dalam graf menggunakan jarak geografis antar titik koordinat yang dihitung menggunakan formula *Haversine*. Penggunaan jarak geografis sebagai bobot menjamin bahwa nilai $g(n)$ dalam algoritma A* mencerminkan jarak sesungguhnya yang telah ditempuh dari titik awal, sehingga hasil pencarian jalur terpendek dalam satuan kilometer bersifat akurat (Zhou & Yang, 2022).

2.3 Implementasi

Implementasi algoritma A* dalam sistem SmartPath terdapat pada berkas *routeFinder.js* di sisi *backend*. Berkas ini memuat tiga fungsi utama: *loadNetwork()* untuk memuat dan menginisialisasi graf jaringan jalan dari berkas *network.geojson*; *snapToRoad(lng, lat)* untuk menemukan titik terdekat pada jaringan jalan; dan *findRoute(startLng, startLat, endLng, endLat)* sebagai fungsi utama yang mengeksekusi pencarian jalur.

Fungsi *loadNetwork()* membaca berkas *network.geojson* yang berisi data jaringan jalan Kota Bengkulu, lalu menginisialisasi objek *PathFinder* dari pustaka *geojson-path-finder* dengan parameter *precision* $1e-5$ untuk menentukan tingkat presisi pencocokan koordinat. Selain itu, fungsi ini juga membangun koleksi titik-titik simpul dari seluruh fitur *LineString* yang ada untuk keperluan *snap-to-road* selanjutnya.

Fungsi *findRoute()* merupakan inti dari sistem pencarian rute. Fungsi ini pertama-tama memanggil *snapToRoad()* untuk kedua titik input, kemudian memanggil *pathFinder.findPath()* yang mengeksekusi algoritma A* secara internal. Hasil pencarian berupa array koordinat yang membentuk jalur terpendek dikembalikan sebagai objek *GeoJSON* dengan tipe *LineString*, disertai dengan

kalkulasi jarak total menggunakan *turf.length()* dan estimasi waktu tempuh berdasarkan asumsi kecepatan rata-rata 40 km/jam.

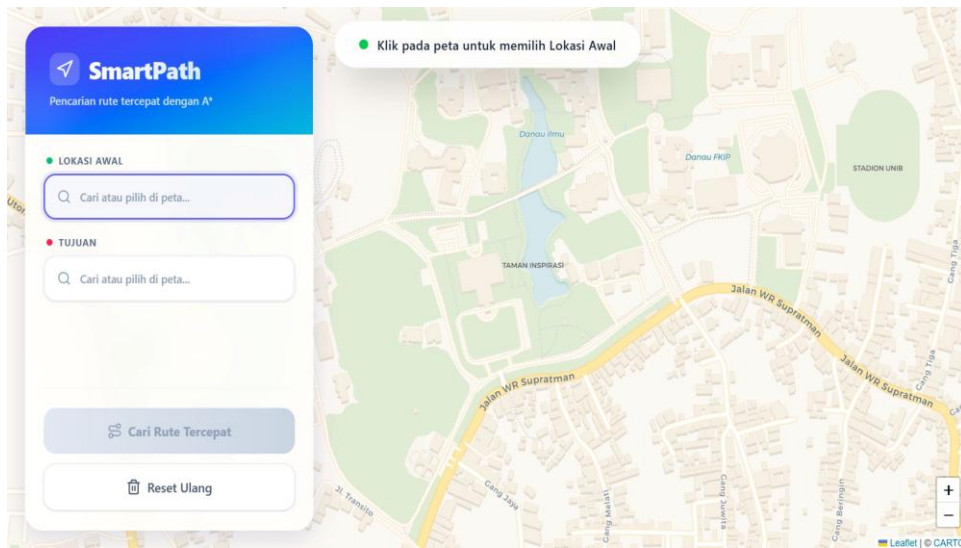
Data jaringan jalan Kota Bengkulu diperoleh melalui Overpass API dengan *bounding box* koordinat -3.80 hingga -3.73 lintang selatan dan 102.24 hingga 102.30 bujur timur. Kueri Overpass QL yang digunakan secara spesifik mengambil seluruh entitas bertipe *way* dengan atribut *highway* yang ada di dalam area tersebut, mencakup berbagai kelas jalan seperti *primary*, *secondary*, *tertiary*, dan *residential*. Data kemudian dikonversi ke format *GeoJSON* dan disimpan sebagai berkas *network.geojson* (OpenStreetMap Contributors, 2023).

Komunikasi antara *frontend* dan *backend* dilakukan melalui REST API dengan *endpoint* POST */api/cari-rute*. *Frontend* mengirimkan koordinat titik awal dan tujuan dalam format JSON, kemudian *backend* merespons dengan objek *GeoJSON* berisi jalur terpendek beserta metadata seperti jarak tempuh, estimasi waktu, dan koordinat hasil *snap-to-road*. Hasil rute tersebut kemudian divisualisasikan sebagai *polyline* berwarna biru tua pada peta Leaflet.

2.4 Antarmuka Pengguna

Antarmuka pengguna sistem SmartPath dirancang dengan pendekatan desain modern yang mengutamakan kemudahan penggunaan. Terdapat dua area utama pada antarmuka, yaitu panel kontrol di sisi kiri dan area peta interaktif di sisi kanan.

Panel kontrol memuat kolom pencarian lokasi awal dan tujuan yang terhubung langsung dengan layanan *geocoding* Nominatim, sehingga pengguna dapat mencari nama lokasi secara teks selain mengeklik langsung pada peta. Setelah rute ditemukan, panel kontrol juga menampilkan informasi rinci hasil pencarian, meliputi jarak tempuh dalam satuan kilometer, estimasi waktu tempuh dalam satuan menit, serta penjelasan singkat mengenai dasar logika pemilihan rute oleh algoritma A*.



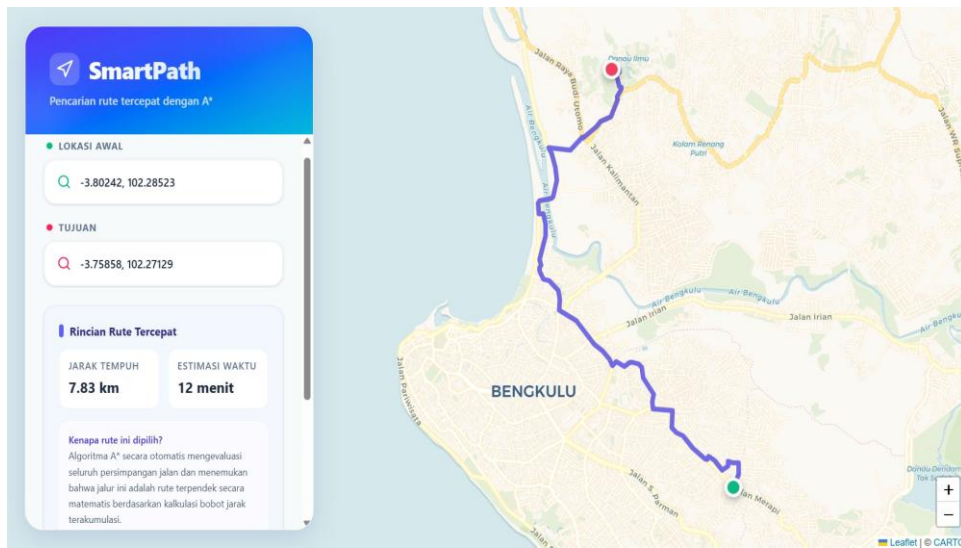
Gambar 1. Tampilan antarmuka awal aplikasi SmartPath

Pada kondisi awal sebelum titik koordinat dipilih, aplikasi menampilkan pesan panduan interaktif di atas peta yang menginstruksikan pengguna untuk mengeklik peta guna memilih lokasi awal. Status pemilihan titik secara otomatis berpindah dari lokasi awal ke tujuan setelah titik awal berhasil dipilih.

2.5 Hasil Pengujian

Pengujian sistem SmartPath dilakukan dengan menggunakan dua titik koordinat pada wilayah Kota Bengkulu, yaitu titik awal pada koordinat -3.80242, 102.28523 dan titik tujuan pada koordinat -3.75858, 102.27129. Kedua koordinat tersebut merepresentasikan lokasi yang berjarak cukup jauh dengan jalur yang melewati beberapa ruas jalan utama Kota Bengkulu, termasuk kawasan Air Bengkulu dan jalan-jalan arteri yang menghubungkan bagian utara dan selatan kota.

Algoritma A* berhasil menemukan jalur terpendek dengan jarak tempuh sebesar 7,83 kilometer dan estimasi waktu tempuh selama 12 menit dengan asumsi kecepatan rata-rata 40 km/jam. Jalur yang ditemukan terlihat mengikuti ruas-ruas jalan utama yang ada, menghindari jalan-jalan kecil yang akan memperpanjang jarak tempuh. Proses pencarian jalur berlangsung sangat cepat dan hasilnya langsung divisualisasikan sebagai garis rute berwarna biru tua pada peta interaktif.



Gambar 2. Hasil pencarian rute terpendek pada peta Kota Bengkulu

Dari hasil visualisasi pada Gambar 2 terlihat bahwa rute yang dihasilkan algoritma A* mengikuti jaringan jalan yang sesungguhnya, bukan garis lurus antara dua titik. Hal ini membuktikan bahwa proses *snap-to-road* dan pencarian jalur pada graf jaringan jalan nyata berfungsi dengan benar. Rute memanfaatkan jalur yang melalui Jalan Air Bengkulu dan beberapa jalan penghubung untuk mencapai tujuan dengan jalur yang paling pendek secara matematis berdasarkan kalkulasi bobot jarak terakumulasi.

Keunggulan algoritma A* dalam sistem ini dibandingkan algoritma lain seperti Dijkstra terletak pada efisiensi eksplorasinya. Dengan memanfaatkan estimasi heuristik berbasis formula *Haversine*, algoritma A* hanya mengeksplorasi simpul-simpul yang secara arah mendekati tujuan, sehingga jumlah simpul yang diproses jauh lebih sedikit dibandingkan algoritma Dijkstra yang mengeksplorasi ke segala arah tanpa terkecuali. Pada jaringan jalan Kota Bengkulu yang memiliki ribuan simpul, perbedaan efisiensi ini sangat signifikan dalam menentukan kecepatan respons sistem (Rachmawati & Gustin, 2020).

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan pembahasan yang telah diuraikan, dapat ditarik beberapa kesimpulan sebagai berikut.

Pertama, algoritma A* terbukti mampu menemukan jalur terpendek secara efektif dan efisien pada jaringan jalan nyata Kota Bengkulu yang direpresentasikan dalam format GeoJSON. Dengan memanfaatkan fungsi heuristik berbasis formula Haversine, algoritma A* mengeksplorasi simpul-simpul secara terarah menuju tujuan sehingga menghasilkan performa pencarian yang lebih unggul dibandingkan algoritma Dijkstra yang bersifat tanpa arah.

Kedua, implementasi sistem SmartPath dengan arsitektur *client-server* terpisah antara *frontend* React.js dan *backend* Node.js berhasil mengintegrasikan algoritma A* dengan data peta nyata dari OpenStreetMap secara fungsional. Mekanisme *snap-to-road* berbasis pustaka Turf.js memastikan bahwa koordinat sembarang yang diberikan pengguna selalu diproyeksikan ke simpul jaringan jalan yang valid sebelum algoritma A* dijalankan.

Ketiga, hasil pengujian menunjukkan bahwa sistem mampu menemukan rute dari koordinat -3.80242, 102.28523 ke koordinat -3.75858, 102.27129 di Kota Bengkulu dengan jarak tempuh 7,83 kilometer dan estimasi waktu 12 menit. Rute yang dihasilkan mengikuti jaringan jalan yang sesungguhnya dan divisualisasikan secara akurat pada peta interaktif berbasis Leaflet.

3.2 Saran

Berdasarkan hasil dan keterbatasan sistem yang ada, terdapat beberapa saran untuk pengembangan lebih lanjut yang dapat dipertimbangkan.

Pertama, sistem dapat dikembangkan dengan menambahkan bobot dinamis pada sisi-sisi graf yang mencerminkan kondisi lalu lintas secara *real-time*. Data kemacetan yang diintegrasikan ke dalam nilai $g(n)$ akan menghasilkan rekomendasi rute yang tidak hanya terpendek secara jarak, tetapi juga optimal dari segi waktu tempuh aktual dalam kondisi lalu lintas yang sedang terjadi.

Kedua, perlu dipertimbangkan penerapan optimasi struktur data antrian prioritas menggunakan *binary heap* atau *Fibonacci heap* pada implementasi algoritma A* agar kompleksitas waktu dapat ditekan lebih lanjut, khususnya ketika sistem digunakan untuk area yang lebih luas dengan jumlah simpul yang jauh lebih besar dari jaringan jalan Kota Bengkulu saat ini.

Ketiga, penambahan fitur pemilihan mode transportasi seperti berjalan kaki, bersepeda, dan berkendara dengan bobot dan batasan jalan yang berbeda akan memperluas fungsionalitas sistem. Selain itu, integrasi fitur *turn restriction* dari data OpenStreetMap akan meningkatkan akurasi rute yang dihasilkan dengan mempertimbangkan larangan belok dan arah jalan satu arah secara lebih komprehensif.

DAFTAR PUSTAKA

- Butler, H., Daly, M., Doyle, A., Gillies, S., Hagen, S., & Schaub, T. (2022). *The GeoJSON format specification and its applications in web-based geospatial systems*. *International Journal of Geographical Information Science*, 36(4), 712–731. <https://doi.org/10.1080/13658816.2022.2034468>
- Cazenave, T., Teytaud, F., & Ung, S. (2023). A* search enhancements for real-world geographic routing problems. *Journal of Artificial Intelligence Research*, 76, 1025–1058. <https://doi.org/10.1613/jair.1.14102>
- Nugraha, A., Sari, M., & Pratama, D. (2021). Implementation of A* algorithm for shortest path finding in geographic information system using OpenStreetMap data. *Journal of Physics: Conference Series*, 1918(1), 012031. <https://doi.org/10.1088/1742-6596/1918/1/012031>
- OpenStreetMap Contributors. (2023). *OpenStreetMap* [Dataset]. OpenStreetMap Foundation. <https://www.openstreetmap.org>
- Rachmawati, D., & Gustin, L. (2020). Analysis of Dijkstra's algorithm and A* algorithm in shortest path problem. *Journal of Physics: Conference Series*, 1566(1), 012061. <https://doi.org/10.1088/1742-6596/1566/1/012061>
- Zhou, B., & Yang, J. (2022). An improved A* algorithm for urban road network routing with dynamic weight adjustment. *IEEE Transactions on Intelligent Transportation Systems*, 23(8), 11432–11445. <https://doi.org/10.1109/TITS.2021.3104823>